

AI TRAINING – DAY 4 ASSIGNMENT

Python Conditional Logic: Decide → Compare → Combine → Nest

Practical + MCQ + Logic Tracing + Debugging + 3 Daily Programs

Level	Beginner → Medium
Suggested Time	3–4 hours
Core Topics	if, elif, else, comparison operators, logical operators, nested conditions
Daily Deliverable	3 completed conditional programs
Main Focus	Learn to design and trace decision logic

Learning Goals

- Understand how Python makes decisions using if, elif and else.
- Use comparison operators correctly: ==, !=, >, <, >= and <=.
- Combine conditions using and, or and not.
- Understand when and why nested conditions are useful.
- Trace a program and predict which branch will execute.
- Identify common conditional-logic errors.
- Build three practical programs independently.
- Explain the logic in your own words.

Assignment Rules

- Write the logic yourself before asking AI for help.
- Before running, predict the output for at least 2 inputs.
- Use meaningful variable names.
- Test boundary values such as 0, 18, 35, 50, 60, 75 and 90 where relevant.
- Avoid giant, unreadable conditions.
- Document Input → Conditions → Output.
- Record one bug you faced and explain how you fixed it.

Part A – Understand the Decision Structure

Structure	Meaning	Your Task
if	Runs a block when a condition is True.	EX:age =20 if age >=19:

		<pre>print("Enter your name:")</pre> output:Enter your name: Explanation:This program stores the value 20 in the variable age. The if statement checks whether the age is greater than or equal to 19. Since 20 >= 19 is True, the program prints "Enter your name:".
if / else	Chooses between two paths.	<pre>EX:marks = 70 if marks >= 90: print("A Grade") else: print("B Grade")</pre> output:B Grade Explanation:This program checks the student's marks. If the marks are 90 or above, it prints "A Grade". Otherwise, it prints "B Grade". Since the marks are 70, the condition is False, so the else block runs and prints "B Grade".
if / elif / else	Chooses among multiple possible paths	<pre>EX:marks = 30 if marks >= 90: print("A Grade") elif marks >= 60: print("B Grade") else: print("Fail")</pre> output:Fail Explanation:First, we check the if condition. If it is false, we check the elif condition. If both conditions are false, the else block runs and prints "Fail"
Nested if	A decision inside another decision.	<pre>marks = 80 if marks >= 50:</pre>

		<pre>if marks >= 75: print("Good marks")</pre> output: Good marks Explanation: This program first checks if the marks are 50 or more. If that is true, it checks if the marks are 75 or more. Since the marks are 80, both conditions are true, so it prints "Good marks".
--	--	---

Part B – Comparison Operators Lab

Predict True or False before running:

- `10 > 5`

Ans: True

- `10 < 5`

Ans: False

- `10 == 10`

Ans: True

- `10 != 10`

Ans: False

- `25 >= 25`

Ans: True

- `24 <= 20`

Ans: False

- `'Python' == 'python'`

Ans: False

Explanation: Python treats capital and small letters as different. So, Python and python are not the same.

- `'AI' != 'ML'`

Ans: True

- `100 == '100'`

Ans: False

Explanation: 100 is a integer

'100' is a string

Python compares both the value and type, so an integer 100 is not equal to the string '100'.

Explain especially why 'Python' == 'python' and 100 == '100' produce their results.

Part C – Logical Operators Lab

Predict the result:

- True and True

Ans:True

- True and False

Ans:False

- False and True

Ans:False

- False and False

Ans:False

- True or False

Ans:True

- False or False

Ans:False

- not True

Ans:False

- not False

Ans:True

Then create your own real-life and Python examples for AND, OR and NOT.

AND:Both conditiond must be true

Example:

```
age=20
```

```
student_id= True
```

```
print(age >=18 and student_id)
```

output:True

OR:At least one condition must be true.

Example:

```
day = "Monday"
```

```
print(day == "Saturday" or day == "Monday")
```

output: True

NOT: Changes the result to the opposite.

Example:

is_going = True

print(not is_going)

output: False

Part D – Decision Tracing

Do not run these first. Predict the output, then test your answer.

```
1)
age = 20
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Ans:

Input: age = 20

Condition checked: 20 >= 18 -> True

Branch executed: if branch

Output: Adult

```
2)
marks = 82
if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
else:
    print("C")
```

Ans:

Input: marks = 82

Condition checked: 82 >= 90 -> False

Next condition: 82 >= 75 -> True

Branch executed: elif branch

Output: B

```
3)
age = 20
has_id = True
if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")
```

Ans:

Input: age = 20, has_id = True

Condition checked: 20 >= 18 and True -> True

Branch executed: if branch

Output: Allowed

```
4)
amount = 5000
if amount >= 10000:
    print("High")
elif amount >= 5000:
    print("Medium")
else:
    print("Low")
```

Ans:

Input: amount = 5000

Condition checked: 5000 >= 5000 -> True

Branch executed: elif branch

Output: Medium

```
5)
username = "admin"
password = "1234"

if username == "admin":
    if password == "1234":
        print("Login successful")
    else:
        print("Wrong password")
else:
    print("Unknown user")
```

Ans:

Input: username = "admin" , password = "1234"

Condition checked: username == "admin" -> True

Second condition: password == "1234" -> True

Output: login successful

For each snippet, write: Input → Condition checked → Branch executed → Output.

Part E – MCQs: Basic

1. 1. Which keyword starts a condition?

[A]

A) if B) when C) check D) condition

1. 2. Which keyword handles another condition after if?

[B]

A) else-if **B) elif** C) otherwise D) next

1. 3. Which keyword handles the remaining case? [B]

A) default **B) else** C) final D) otherwise

1. 4. Which operator checks equality? [B]

A) = **B) ==** C) === D) equals

1. 5. What does > mean? [C]

A) less than B) equal **C) greater than** D) not equal

1. 6. Which operator means AND? [B]

A) & **B) and** C) && D) plus

1. 7. Which operator means OR? [A]

A) or B) || C) | D) either

1. 8. Which operator reverses a Boolean result? [B]

A) reverse **B) not** C) ! D) opposite

1. 9. In a single if/elif/else chain, how many else blocks are normally used? [B]

A) 0 only **B) exactly one** C) many D) two

1. 10. Can an if statement exist inside another if statement? [B]

A) No **B) Yes** C) Only in loops D) Only in functions

Part F – MCQs: Medium

1. If 10 > 5, what is printed by if 10 > 5: print('Yes')? **A) Yes** B) No C) Error D) True [A]

2. Which is a correct age condition? **A) if age >= 18:** B) if age == 18: C) if age = 18: D) if age greater 18: [A]

3. In an if/elif chain, what happens after the first True condition? A) All branches run **B) The first True branch runs and the chain stops** C) else always runs D) Program restarts [B]

4. What is 10 > 5 and 2 > 8? A) True **B) False** C) 10 D) Error [B]

5. What is 10 > 5 or 2 > 8? **A) True** B) False C) 8 D) Error [A]

6. What is not(10 > 5)? A) True **B) False** C) None D) Error [B]

7. Which represents age between 18 and 60 inclusive? **A) age >= 18 and age <= 60** B) age >= 18 or age <= 60 C) age > 18 and age > 60 D) age < 18 and age > 60 [A]

8. What is a nested condition? A) A loop inside a string **B) A condition inside another condition** C) Two variables D) A function call [B]

9. Nested conditions are useful when: **A) The second decision depends on the first** B) You want no conditions C) Only for printing D) Only for loops [A]

10. A common grade-boundary error is: **A) Using incorrect >= or > conditions** B) Using print C) Using variables D) Using input [A]

Part G – MCQs: Thinking

1. Why does elif order matter? **A) Python checks from top to bottom and stops at the first True branch** B) It does not matter C) It sorts conditions D) It runs randomly [A]
2. If A >= 90 and B >= 75, which should be checked first? A) >= 75 **B) >= 90** C) < 75 D) == 90 [A]
3. Adult AND has ID can be written as: **A) age >= 18 and has_id** B) age >= 18 or has_id C) age <= 18 and has_id D) not age [A]
4. 'Not a student' can be represented by: A) is_student **B) not is_student** C) is_student == True D) student not [B]
5. A menu option 9 should be: A) Ignored **B) Handled explicitly or rejected as invalid** C) Always option 1 D) Automatically accepted [B]
6. Why test boundary values? **A) To find mistakes around >=, >, <= and <** B) To make code longer C) To avoid input D) To remove variables [A]
7. Why is age >= 18 or age <= 60 usually wrong for a range? **A) It is true for almost every number** B) It always returns False C) or cannot be used D) Python does not support ranges [A]
8. When is nested if more readable? **A) When the second decision depends on the first** B) Always C) Never D) Only with strings [A]
9. If many branches become hard to read, what should you do? A) Make one giant condition **B) Break logic into smaller understandable steps** C) Remove tests D) Use random conditions [B]
10. A grade calculator should test: A) Only 100 **B) Normal, boundary and invalid cases** C) Only one mark D) No invalid input [B]

Part H – Daily Deliverable: 3 Conditional Programs

Program 1 – Grade Calculator

- Ask for student name and marks.
- Validate marks from 0 to 100.
- Use if/elif/else with at least 5 grade categories.
- Display the grade and a meaningful message.
- Test 90, 75, 60, 50 and 0.
- Test invalid values such as -5 and 110.

PROGRAM:

```
name = input("Enter student name: ")
```

```
marks = int(input("Enter marks: "))
```

```
if marks < 0 or marks > 100:
```

```
    print("Invalid marks for %s! Please enter a value between 0 and 100." % name)
```

```
elif marks >= 90:
```

```
    print("Student: %s | Grade: A | Message: Excellent work!" % name)
```

```
elif marks >= 75:
```

```
    print("Student: %s | Grade: B | Message: Good job!" % name)
```

```
elif marks >= 60:
```

```
    print("Student: %s | Grade: C | Message: You passed." % name)
```

```
elif marks >= 50:
```

```
    print("Student: %s | Grade: D | Message: Needs improvement." % name)
```

```
else:
```

```
    print("Student: %s | Grade: F | Message: Failed. Please try again." % name)
```

output:

Test 1: 90

Enter student name: Lasya Kolluru

Enter marks: 90

Student: Lasya Kolluru | Grade: A | Message: Excellent work!

Test 2: 75

Enter student name: Lasya Kolluru

Enter marks: 75

Student: Lasya Kolluru | Grade: B | Message: Good job!

Test 3: 60

Enter student name: Lasya Kolluru

Enter marks: 60

Student: Lasya Kolluru | Grade: C | Message: You passed.

Test 4: 50

Enter student name: Lasya Kolluru

Enter marks: 50

Student: Lasya Kolluru | Grade: D | Message: Needs improvement.

Test 5: 0

Enter student name: Lasya Kolluru

Enter marks: 0

Student: Lasya Kolluru | Grade: F | Message: Failed. Please try again.

Test 6: -5 (Invalid)

Enter student name: Lasya Kolluru

Enter marks: -5

Invalid marks for Lasya Kolluru! Please enter a value between 0 and 100.

Test 7: 110 (Invalid)

Enter student name: Lasya Kolluru

Enter marks: 110

Invalid marks for Lasya Kolluru! Please enter a value between 0 and 100

Program 2 – Eligibility Checker

Choose one realistic scenario: scholarship, course admission, job interview, driving eligibility, loan/application, or gym membership.

- Use at least 3 inputs.
- Use at least one and condition.
- Use at least one or condition.
- Create at least 3 possible outcomes.
- Write your eligibility rules clearly before coding.
- Test a qualifying case, a non-qualifying case and a boundary case.

PROGRAM:

```
name = input("Enter student name: ")

gpa = float(input("Enter GPA (0.0 to 10.0): "))

attendance = int(input("Enter attendance percentage (0 to 100): "))

sports = input("Do you have a sports quota? (yes/no): ").strip().lower()

if gpa >= 9.0 and attendance >= 80:

    print("Result for %s: Eligible for Full Scholarship!" % name)

elif gpa >= 8.0 and (attendance >= 75 or sports == "yes"):

    print("Result for %s: Eligible for Partial Scholarship!" % name)

else:

    print("Result for %s: Not Eligible for a scholarship." % name)
```

Output:

Test 1: Qualifying Case (High GPA and High Attendance)

```
Enter student name: Lasya Kolluru
Enter GPA (0.0 to 10.0): 9.2
Enter attendance percentage (0 to 100): 85
Do you have a sports quota? (yes/no): no
Result for Lasya Kolluru: Eligible for Full Scholarship!
```

Test 2: Boundary Case (Exact 8.0 GPA, Low Attendance, but has Sports Quota)

```
Enter student name: Lasya Kolluru
Enter GPA (0.0 to 10.0): 8.0
Enter attendance percentage (0 to 100): 60
Do you have a sports quota? (yes/no): yes
Result for Lasya Kolluru: Eligible for Partial Scholarship!
```

Test 3: Non-Qualifying Case (Low GPA, even with High Attendance)

```
Enter student name: Lasya Kolluru
Enter GPA (0.0 to 10.0): 7.5
Enter attendance percentage (0 to 100): 95
Do you have a sports quota? (yes/no): no
Result for Lasya Kolluru: Not Eligible for a scholarship.
```

Program 3 – Simple Menu

- Create at least 4 menu choices.
- Ask the user for a choice.
- Use if/elif/else.
- Handle invalid choices.
- Use at least one nested if.
- Suggested project: Banking Menu — Check Balance, Deposit, Withdraw, Exit. For withdrawal, use a nested condition to check sufficient balance.
- Test every option and at least one invalid option.

PROGRAM:

```
balance = 5000

account_name = "Lasya Kolluru"

print("Welcome to Simple Bank, %s!" % account_name)
```

```

print("1. Check Balance")

print("2. Deposit")

print("3. Withdraw")

print("4. Exit")

choice = input("Enter your choice (1-4): ")

if choice == '1':

    print("Your current balance is: ₹%d" % balance)

elif choice == '2':

    deposit_amount = int(input("Enter amount to deposit: "))

    balance = balance + deposit_amount

    print("Deposit successful. Your new balance is: ₹%d" % balance)

elif choice == '3':

    withdraw_amount = int(input("Enter amount to withdraw: "))

    # Nested if condition to check sufficient balance

    if withdraw_amount <= balance:

        balance = balance - withdraw_amount

        print("Withdrawal successful. Your new balance is: ₹%d" % balance)

    else:

        print("Transaction failed! Insufficient balance. You only have: ₹%d" %
balance)

elif choice == '4':

    print("Thank you for using Simple Bank, %s. Goodbye!" % account_name)

else:

    print("Invalid choice! Please select a valid option from 1 to 4.")

```

OUTPUT:

Test 1: Choice 1 (Check Balance)

Enter your choice (1-4): 1

Your current balance is: ₹5000

Test 2: Choice 2 (Deposit)

Enter your choice (1-4): 2

Enter amount to deposit: 2000

Deposit successful. Your new balance is: ₹7000

Test 3: Choice 3 (Withdraw - Sufficient Balance)

Enter your choice (1-4): 3

Enter amount to withdraw: 1000

Withdrawal successful. Your new balance is: ₹4000

Test 4: Choice 3 (Withdraw - Insufficient Balance)

Enter your choice (1-4): 3

Enter amount to withdraw: 8000

Transaction failed! Insufficient balance. You only have: ₹5000

Test 5: Choice 4 (Exit)

Enter your choice (1-4): 4

Thank you for using Simple Bank, Lasya Kolluru. Goodbye!

Test 6: Invalid Choice

Enter your choice (1-4): 7

Invalid choice! Please select a valid option from 1 to 4.

Part I – Debugging Lab

For every bug: identify the problem, explain why it happens, fix it, and test it.

```
Bug 1
age = 18
if age = 18:
    print("Adult")
```

Hint: What is the difference between = and ==?

```
code:age = 18
      if age == 18:
          print("Adult")
```

Output:Adult

Explanation:--> assign a value to the variable

== -> compares two values

```
Bug 2
marks = 85
if marks >= 50:
    print("Excellent")
elif marks >= 80:
    print("Pass")
```

Why does Excellent never print? Fix the order of conditions.

code:

```
marks = 85
if marks >= 80:
    print("Pass")
elif marks >= 50:
    print("Excellent")
```

output:pass

Explanation:the marks are 85, the first condition marks >= 80 is true, so Python prints "Pass" and does not check the elif condition.

```
Bug 3
age = 25
if age >= 18 or age <= 60:
    print("Eligible")
```

Explain why this is wrong for an 18–60 range and correct it using and.

code:age = 25

```
if age >= 18 and age <= 60:

    print("Eligible")
```

output:Eligible

Explanation: we use and because both conditions need to be true. If we use or all ages are eligible even out of range. Therefore and is correct for checking an age range from 18 to 60.

Since 25 is greater than or equal to 18 and less than or equal to 60, the condition is True, so Python prints "Eligible".

```
Bug 4
has_id = True
age = 17

if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")
```

Predict the output and explain which part of the condition makes it False.

code:has_id = True

```
age = 17

if age >= 18 and has_id:

    print("Allowed")

else:

    print("Not allowed")
```

output:Not allowed

Explanation: The condition is false because `age >= 18` is false .although `has_id` is true and require both conditions need to be true

```
Bug 5
choice = 2

if choice == 1:
    print("Deposit")
elif choice == 2:
    print("Withdraw")
elif choice == 2:
    print("Check Balance")
```

Why is the final branch unreachable? Redesign the menu conditions.

code:`choice = 2`

`if choice == 1:`

`print("Deposit")`

`elif choice == 2:`

`print("Withdraw")`

`elif choice == 3:`

`print("Check Balance")`

output:`Withdraw`

Explanation:The final branch was unreachable because `choice == 2` was repeated. When choice is 2, Python executes the first matching condition and skips the remaining elif conditions. I changed the final condition to `choice == 3`, so the menu can correctly display 'Check Balance'."

```
Bug 6
password = "1234"

if password == "1234":
    if password == "admin":
        print("Success")
    else:
        print("Wrong")
```

Explain the nested logic. Why can this never print Success? Redesign the login logic sensibly.

code:`username = "admin"`

`password = "1234"`

`if username == "admin" and password == "1234":`

`print("Success")`

`else:`

`print("Wrong username or password")`

output:Success

Explanation: The original code asks if the password is "1234", and then immediately asks if that exact same password is "admin". One variable cannot be two different things at the same time. I fixed it by creating a separate username variable so the program can check the username and password properly.

Part J – Nested Conditions Challenge

Build a Login + Access Control program.

- Ask for username and password.
- If the username is correct, use a nested condition to check the password.
- If both are correct, ask whether the user is an admin.
- Show different messages for normal users and admins.
- Show separate messages for wrong username and wrong password.
- Test at least 5 combinations.

code:

```
correct_user = "lasya"
correct_pass = "lasya123"
username = input("Enter username: ")
password = input("Enter password: ")
if username == correct_user:
    if password == correct_pass:
        admin_check = input("Are you an admin? (yes/no): ").strip().lower()
        if admin_check == "yes":
            print("Login successful! Welcome Admin %s. You have full system controls." % username)
        else:
            print("Login successful! Welcome User %s. You have standard access." % username)
    else:
        print("Login failed! Incorrect password for user %s." % username)
else:
    print("Login failed! Username '%s' not found in our system." % username)
```

OUTPUT:

Test 1: Correct Username, Correct Password, Is Admin

Enter username: lasya

Enter password: lasya123

Are you an admin? (yes/no): yes

Login successful! Welcome Admin lasya. You have full system controls.

Test 2: Correct Username, Correct Password, Not Admin

Enter username: lasya

Enter password: lasya123

Are you an admin? (yes/no): no

Login successful! Welcome User lasya. You have standard access.

Test 3: Correct Username, Wrong Password

Enter username: lasya

Enter password: wrongpassword

Login failed! Incorrect password for user lasya.

Test 4: Wrong Username, Any Password

Enter username: john

Enter password: lasya123

Login failed! Username 'john' not found in our system.

Test 5: Wrong Username, Wrong Password

Enter username: admin

Enter password: adminpassword

Login failed! Username 'admin' not found in our system.

Part K – Real-World Logic Challenge

Choose one: food-delivery charge calculator, movie-ticket pricing, college-attendance eligibility, ATM withdrawal, electricity-bill category, or mobile-data plan.

- Use at least 4 inputs or decision factors.
- Create at least 5 possible outcomes.
- Use comparison and logical operators.
- Use at least one nested condition.
- Draw a decision tree before coding.
- Test normal, boundary and invalid cases.

Program:

```
customer_name = "Lasya Kolluru"

age = int(input("Enter age: "))

day = input("Enter day type (weekday/weekend): ").strip().lower()

seat = input("Enter seat type (standard/vip): ").strip().lower()

member = input("Do you have a membership? (yes/no): ").strip().lower()

if age < 0 or age > 120:

    print("Hello %s, invalid age entered! Please try again." % customer_name)

elif age <= 12:

    print("Hello %s, your ticket is a Child ticket. Price: ₹100" % customer_name)

elif age >= 60:

    print("Hello %s, your ticket is a Senior ticket. Price: ₹150" % customer_name)

else:

    if seat == "vip":

        if member == "yes" and day == "weekday":

            print("Hello %s, you get the VIP Member Weekday discount! Price: ₹300" % customer_name)

        else:

            print("Hello %s, your ticket is a Full VIP ticket. Price: ₹400" % customer_name)

    else:

        print("Hello %s, your ticket is a Standard Adult ticket. Price: ₹200" % customer_name)
```

OUTPUT:**Test 1:** Normal Case (Standard Adult)

```
Enter age: 25

Enter day type (weekday/weekend): weekend

Enter seat type (standard/vip): standard

Do you have a membership? (yes/no): no

Hello Lasya Kolluru, your ticket is a Standard Adult ticket. Price: ₹200
```

Test 2: Boundary Case (Exactly age 12 - Child)

```
Enter age: 12

Enter day type (weekday/weekend): weekday
```

Enter seat type (standard/vip): standard

Do you have a membership? (yes/no): no

Hello Lasya Kolluru, your ticket is a Child ticket. Price: ₹100

Test 3: Boundary Case (Exactly age 60 – Senior)

Enter age: 60

Enter day type (weekday/weekend): weekend

Enter seat type (standard/vip): vip

Do you have a membership? (yes/no): yes

Hello Lasya Kolluru, your ticket is a Senior ticket. Price: ₹150

Test 4: Nested Logic Success (VIP Member Weekday)

Enter age: 30

Enter day type (weekday/weekend): weekday

Enter seat type (standard/vip): vip

Do you have a membership? (yes/no): yes

Hello Lasya Kolluru, you get the VIP Member Weekday discount! Price: ₹300

Test 5: Nested Logic Fail (VIP Member on Weekend)

Enter age: 30

Enter day type (weekday/weekend): weekend

Enter seat type (standard/vip): vip

Do you have a membership? (yes/no): yes

Hello Lasya Kolluru, your ticket is a Full VIP ticket. Price: ₹400

Test 6: Invalid Case (Negative Age)

Enter age: -5

Enter day type (weekday/weekend): weekday

Enter seat type (standard/vip): standard

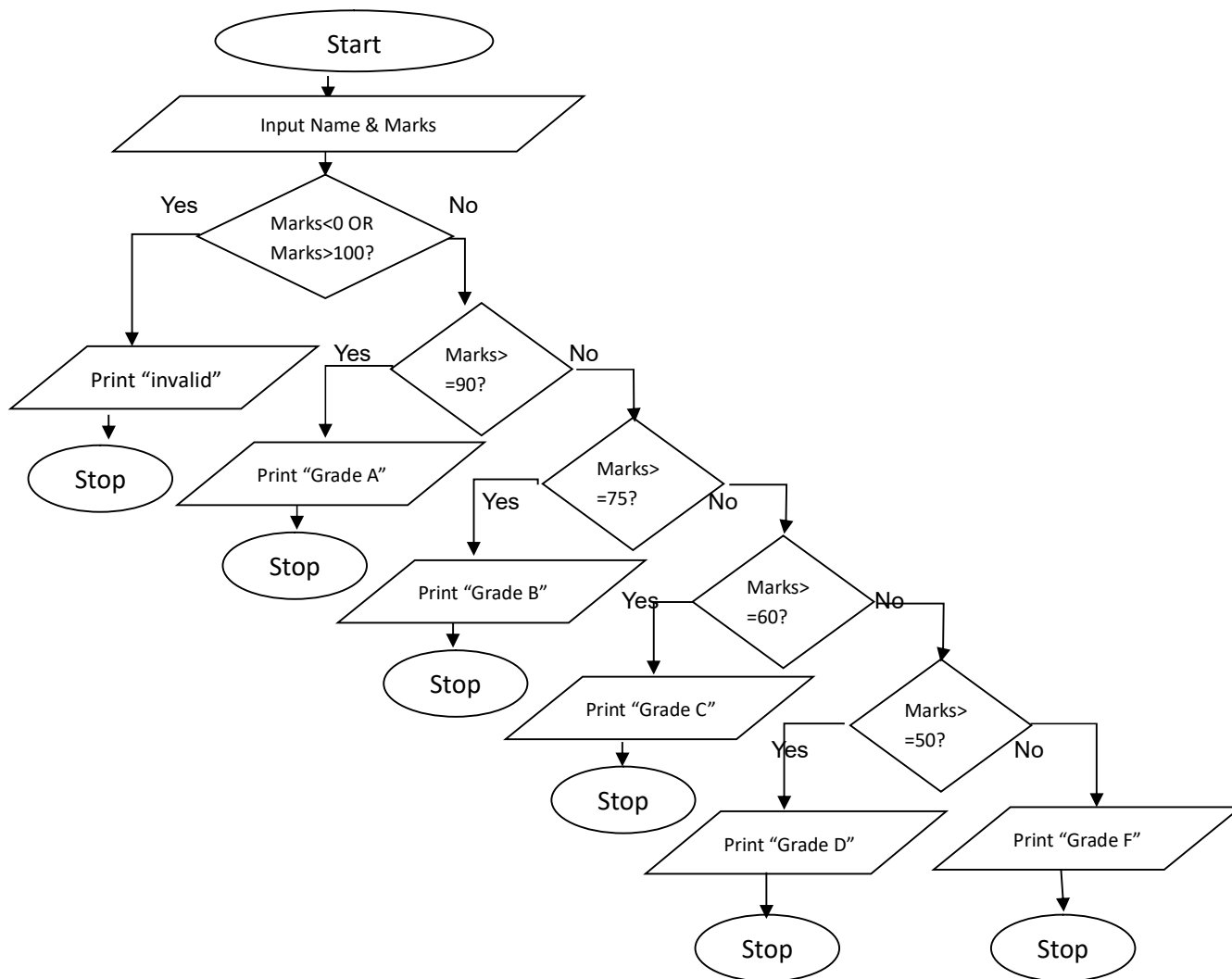
Do you have a membership? (yes/no): no

Hello Lasya Kolluru, invalid age entered! Please try again.

Part L – Draw a Decision Tree

Choose the Grade Calculator or Eligibility Checker.

- Start → Input → First decision → Branches → Next decisions → Final output.
- Your flowchart must match the code you finally submit.
- After coding, compare your flowchart with the actual program.



Part M – Explain Your Code

1. Why did you use if/elif/else instead of only if statements?

Using just if makes the computer check every single rule and wastes time. elif is faster because it stops checking once it finds the right answer.

2. Where did you use comparison operators?

I used them to compare numbers. For example, `age <= 12` checks if someone is a child, and `marks >= 90` checks for an A grade.

3. Where did you use and/or/not?

I used or to catch bad inputs (like an age below 0). I used and when two things must happen together, like needing a membership AND coming on a weekday.

4. Where did you use a nested condition?

I used it in the movie program. Inside the adult ticket section, I put another if block to check for VIP seats and discounts.

5. Which boundary value caused the most thinking?

The exact ages of 12 and 60. I had to use `<=` and `>=` carefully so a person exactly 12 or exactly 60 gets the correct ticket price.

6. What bug did you encounter?

At first, members were getting a weekday discount on the weekend by mistake.

7. How did you improve your first version?

I changed an `or` to an `and`. This forced the computer to make sure BOTH the membership and the weekday were true before giving the discount.

8. Which condition would be hardest for another developer to understand?

The VIP discount rule. Since it is hidden inside other `if` blocks and checks two things at once, someone else might have to read it twice to understand it.

Part N – Self Check

Skill	Yes / Need Practice	Notes
Basic <code>if</code>	Yes	I know how to run code when a rule is true.
<code>if/elif/else</code>	Yes	I can check multiple rules in order.
Using <code>==</code> correctly	Yes	I know <code>==</code> compares and <code>=</code> assigns.
<code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	Yes	I can compare numbers for ages and marks.
<code>and/or</code>	Yes	I know <code>and</code> needs both true, or <code>or</code> needs just one.
<code>not</code>	Yes	I know it flips the true/false result.
Nested conditions	Yes	I can put an <code>if</code> inside another <code>if</code> .
Tracing conditional programs	Yes	I can predict the output before running the code.
Invalid input/choices	Yes	I can catch wrong inputs like negative numbers.
Boundary testing	Yes	I successfully tested exact edge numbers like 12 and 60.
Completed 3 required programs	Yes	Built the Grade, Eligibility, and Menu programs.
Can explain own logic	Yes	I explained my code clearly in simple words.

Submission Checklist

- Comparison operator experiments completed.
- Logical operator experiments completed.
- Decision tracing completed.
- All 30 MCQs answered.

- Grade Calculator completed.
- Eligibility Checker completed.
- Simple Menu completed.
- Debugging Lab completed.
- Nested Login + Access Control challenge completed.
- Decision tree submitted.
- Reflection and self-check completed.

Suggested Evaluation

Area	Weight
Concepts + MCQs	20%
3 Daily Programs	35%
Debugging	15%
Nested Challenge	15%
Testing + Explanation	15%

DAY 4 PRINCIPLE

*Don't just write conditions. Learn to think in decisions:
What is the input? What are the rules? What happens in every possible case?*